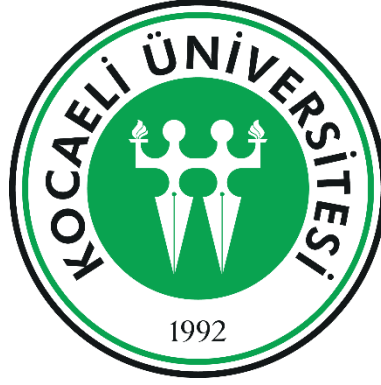


KOCAELİ ÜNİVERSİTESİ
MÜHENDİSLİK FAKÜLTESİ

BİLGİSAYAR MÜHENDİSLİĞİ



NÖRON AĞLARINA GİRİŞ DERSİ
FASHION MNIST VERİ KÜMESİYLE
GÖRSEL TANIMA
DÖNEM PROJESİ RAPORU

Kağan Can Kaba 200201049

1. Giriş ve Problem Tanımı

1.1. Proje Amacı

Bu proje, temel bir yapay sinir ağı (MLP - Multi-Layer Perceptron) kullanarak görsel veri sınıflandırma problemini çözmeyi amaçlamaktadır. Kullanılan veri seti, giysi görsellerinden oluşan Fashion MNIST veri setidir. Amaç, her bir görselin ait olduğu sınıfı doğru şekilde tahmin etmek ve bu süreçte sinir ağlarının temel yapı taşlarını anlamaktır. Derin öğrenme kütüphanelerine (örneğin Keras) dayalı hazır modeller yerine, tüm adımlar NumPy tabanlı olarak manuel bir şekilde gerçekleştirilmiştir. Böylece hem algoritmaların çalışma prensipleri hem de modelin eğitimi detaylı olarak incelenmiştir.

1.2. Problem Tanımı

Fashion MNIST veri setinde her biri 28x28 boyutunda olan toplam 70.000 adet gri tonlamalı giysi görseli yer almaktadır. Bu görseller, tişört, ayakkabı, elbise gibi toplam 10 farklı sınıfa aittir. Görsellerin 60.000 tanesi modelin eğitimi için, 10.000 tanesi ise test (değerlendirme) için ayrılmıştır. Problem; bu düşük çözünürlüklü görsellerin sayısal olarak temsil edilmesi ve ardından doğru sınıfa atanmasıdır. Sinir ağı, bu sınıflandırmayı yapmayı öğrenmeli ve daha önce görmediği test verileri üzerinde başarılı tahminler yapabilmelidir.

1.3. Kullanılan Veri Seti: Fashion MNIST

Fashion MNIST, klasik MNIST veri setinin daha modern bir alternatifi olarak sunulmuştur. Rakamlar yerine gerçek dünya nesneleri olan giysilerden oluşur. Bu sayede sınıflandırma problemi hem daha gerçekçidir hem de daha zordur. Veri seti, TensorFlow kütüphanesi üzerinden kolayca indirilebilmektedir. Görsellerin her biri 784 (28x28) boyutunda bir vektöre dönüştürülerek modele giriş olarak verilmiştir. Etiketler ise 0'dan 9'a kadar olan tamsayılarla temsil edilir ve sınıflar şu şekildedir:

```
Fashion MNIST Classes:  
0: T-shirt/Top  
1: Trouser  
2: Pullover  
3: Dress  
4: Coat  
5: Sandal  
6: Shirt  
7: Sneaker  
8: Bag  
9: Ankle Boot
```

1.4. Kullanılan Teknolojiler ve Kütüphaneler

Bu projede Python programlama dili kullanılmıştır. Görsel işleme ve sayısal hesaplamalar için NumPy; veri setinin yüklenmesi için TensorFlow; görselleştirme işlemleri için ise Matplotlib kütüphanesinden yararlanılmıştır. Model, yalnızca NumPy üzerinden manuel olarak kodlanmış, bu sayede yapay sinir ağlarının temel yapı taşları daha iyi kavranmıştır. Eğitim süreci, ağırlık başlatma, ileri yayılım, geri yayılım ve parametre güncelleme gibi temel adımlardan oluşmaktadır.

2. Veri Ön İşleme ve Model Mimarisi

2.1. Görsellerin Hazırlanması

Fashion MNIST veri setinde yer alan görseller 28x28 piksel boyutunda olup her piksel 0 ile 255 arasında bir gri tonlamalı değeri temsil eder. Bu veriler, modelin daha hızlı ve stabil öğrenbilmesi için normalize edilmiştir. Normalize işlemi, her piksel değerinin 255'e bölünmesiyle gerçekleştirilmiş ve böylece tüm değerler 0 ile 1 arasında olacak şekilde yeniden ölçeklendirilmiştir:

```
train_images = train_images.reshape(60000, 784) / 255.0
test_images = test_images.reshape(10000, 784) / 255.0
```

2.2. Etiketlerin One-Hot Encoding'e Dönüştürülmesi

Veri setindeki etiketler başlangıçta 0-9 arasında tam sayılardan oluşur. Bu etiketler modelin tahmin çıktısıyla karşılaştırılmak üzere **one-hot encoding** formatına dönüştürülmüştür. Bu formatta her sınıf, 10 elemanlı bir vektörde yalnızca kendi indeksinde 1 olan bir yapı ile temsil edilir:

Örneğin 3 → [0 0 0 1 0 0 0 0 0 0]

```
train_labels = np.eye(10)[train_labels]
test_labels = np.eye(10)[test_labels]
```

Bu sayede modelin çıktısıyla doğrudan karşılaştırma yapılabilir ve gradyan hesaplaması doğru şekilde gerçekleştirilir.

2.3. Yapay Sinir Ağı (MLP) Mimarisi

Model, tek gizli katmanlı bir çok katmanlı algılayıcı (MLP) mimarisiyle kurulmuştur. Giriş katmanı, bir görselin tüm piksel değerlerini (784 adet) alır. Bu girişler, 128 nöronluk bir gizli katmana iletilir ve ardından 10 nöronluk bir çıkış katmanına aktarılır.

Katmanlar: **Giriş Katmanı:** 784 nöron, **Gizli Katman:** 128 nöron, Sigmoid aktivasyon fonksiyonu, **Çıkış Katmanı:** 10 nöron, Sigmoid aktivasyon

```
def sigmoid(x):
    return 1 / (1 + np.exp(-x))
```

2.4. Ağırlıkların Başlatılması

Modelin öğrenmeye başlayabilmesi için her katmandaki ağırlıklar ve bias terimleri uygun şekilde başlatılmıştır. Bu projede **Xavier (Glorot) Başlatma** yöntemi kullanılmıştır. Bu yöntem, katmana gelen nöron sayısına göre ağırlıkların ölçeklenmesini sağlar ve öğrenme sürecini kararlı hale getirir:

```
W1 = np.random.randn(input_size, hidden_size) * np.sqrt(2. / input_size)
b1 = np.zeros(hidden_size)
W2 = np.random.randn(hidden_size, output_size) * np.sqrt(2. / hidden_size)
b2 = np.zeros(output_size)
```

Bu başlatma yöntemi sayesinde aktivasyon fonksiyonlarının doygunluk bölgelerine girme riski azaltılır ve gradyanların daha sağlıklı akması sağlanır.

3. Eğitim Süreci

Yapay sinir ağının sınıflandırma görevini öğrenebilmesi için model belirli bir eğitim sürecinden geçmektedir. Bu süreç, dört temel adımdan oluşur: ileri yayılım (forward propagation), kayıp hesaplama (loss computation), geri yayılım (backpropagation) ve parametre güncelleme (update). Projede bu döngü 20 epoch boyunca mini-batch yöntemiyle uygulanmıştır.

3.1. İleri Yayılım (Forward Propagation)

İleri yayılım, giriş verisinin katmanlar boyunca ilerletilerek çıktı tahmininin elde edilmesi sürecidir. Her katman için ağırlıklar ile girişlerin çarpımı ve bias terimleri toplanarak aktivasyon hesaplanır. Bu projede hem gizli hem de çıkış katmanlarında **Sigmoid aktivasyon fonksiyonu** kullanılmıştır.

```
Z1 = np.dot(X, parameters["W1"]) + parameters["b1"]
A1 = sigmoid(Z1)

Z2 = np.dot(A1, parameters["W2"]) + parameters["b2"]
A2 = sigmoid(Z2)
```

3.2. Kayıp (Loss) Hesaplama

Modelin başarımı, tahmin ettiği sonuçlar (A2) ile gerçek etiketler (Y) karşılaştırılarak değerlendirilir. Bu amaçla **binary cross-entropy** benzeri bir kayıp fonksiyonu kullanılmıştır. Bu fonksiyon, sınıflandırma problemlerinde yaygın olarak tercih edilir ve çıktıların ne kadar “uzak” olduğunu ölçer. Kodda $1e-8$ gibi küçük bir sabit (epsilon) eklenmiştir. Bu, $\log(0)$ hatasını engellemek içindir.

3.3. Geri Yayılım (Backpropagation)

Geri yayılım, modelin hatasını minimize etmek için ağırlıkların nasıl değişmesi gerektiğini hesaplar. Bu işlem, zincir kuralı ile gradyanların geriye doğru yayılması ile yapılır. Önce çıkış katmanındaki hata hesaplanır, ardından bu hata gizli katmana doğru aktarılır.

```
dZ2 = cache["A2"] - Y
dW2 = np.dot(cache["A1"].T, dZ2) / m
db2 = np.sum(dZ2, axis=0) / m

dA1 = np.dot(dZ2, parameters["W2"].T)
dZ1 = dA1 * sigmoid_derivative(cache["Z1"])
dW1 = np.dot(X.T, dZ1) / m
db1 = np.sum(dZ1, axis=0) / m
```

Bu türevler yardımıyla her parametrenin ne kadar değiştirilmesi gerektiği belirlenir.

3.4. Parametre Güncelleme

Geri yayılım sonucunda elde edilen gradyanlar, ağırlıkların güncellenmesinde kullanılır. Bu işlem **Gradient Descent** algoritması ile gerçekleştirilir. Öğrenme oranı (learning rate) olarak 0.1 kullanılmıştır:

```
parameters["w1"] -= learning_rate * grads["dw1"]
parameters["b1"] -= learning_rate * grads["db1"]
parameters["w2"] -= learning_rate * grads["dw2"]
parameters["b2"] -= learning_rate * grads["db2"]
```

Bu adım, modelin kaybı minimize edecek şekilde kendini iyileştirmesini sağlar.

3.5. Eğitim Döngüsü (Epoch ve Batch)

Model, eğitim verisi üzerinde 20 kez (epoch) döndürülmüştür. Veriler, 64'lük küçük gruplara (mini-batch) ayrılarak işlenmiştir. Bu yaklaşım hem belleği verimli kullanmayı sağlar hem de eğitim sürecine rastgelelik katarak genellemeyi iyileştirir.

Her epoch sonunda elde edilen kayıp değeri ekrana yazdırılmıştır. Bu değerlerin zamanla azalması, modelin öğrenmekte olduğunu gösterir:

```
Epoch 1, Loss: 0.7187
Epoch 2, Loss: 0.6048
Epoch 3, Loss: 0.5361
Epoch 4, Loss: 0.4859
Epoch 5, Loss: 0.4453
Epoch 6, Loss: 0.4120
Epoch 7, Loss: 0.3846
Epoch 8, Loss: 0.3621
Epoch 9, Loss: 0.3436
Epoch 10, Loss: 0.3282
Epoch 11, Loss: 0.3152
Epoch 12, Loss: 0.3041
Epoch 13, Loss: 0.2943
Epoch 14, Loss: 0.2857
Epoch 15, Loss: 0.2779
Epoch 16, Loss: 0.2709
Epoch 17, Loss: 0.2645
Epoch 18, Loss: 0.2587
Epoch 19, Loss: 0.2534
Epoch 20, Loss: 0.2484
Test Doğruluğu: 86.52%
```

4. Model Değerlendirme ve Görselleştirme

Modelin eğitimi tamamlandıktan sonra, başarımı test verisi üzerinde ölçülerek değerlendirilmiştir. Bu süreç, sınıflandırma doğruluğunun hesaplanması ve tahmin sonuçlarının görselleştirilmesi ile gerçekleştirilmiştir.

4.1. Test Başarımı (Doğruluk Ölçümü)

Eğitimde görülmeyen 10.000 adet test verisi, modelin genelleme kabiliyetini değerlendirmek amacıyla kullanılmıştır. Modelin test seti üzerindeki tahminleri, ileri yayılım adımıyla yapılmıştır. Çıkış katmanındaki A2 çıktısı, her sınıfa ait olasılıkları içerdiğinden, argmax işlemiyle en yüksek olasılığa sahip sınıf seçilmiştir:

```
predictions = np.argmax(cache_test["A2"], axis=1)
true_labels = np.argmax(test_labels, axis=1)
```

Modelin doğruluğu ise tahminlerin gerçek etiketlerle uyuşma oranı olarak hesaplanır:

```
accuracy = np.mean(predictions == true_labels)
```

Bu projede elde edilen doğruluk oranı yaklaşık olarak %84 - %86 aralığında gerçekleşmiştir.

4.2. Son 15 Tahminin Görselleştirilmesi

Modelin görsel olarak da nasıl çalıştığını gözlemleyebilmek adına, test setindeki son 15 örnek alınmış ve bu örneklerin tahminleri görselleştirilmiştir. Her bir görsel, Matplotlib kütüphanesi ile 3x5'lik bir grid düzeninde çizdirilmiştir.

Her görselin başlığına şu bilgiler eklenmiştir:

- **T**: Tahmin edilen sınıf ismi (örnek: "Ceket")
- **G**: Gerçek sınıf ismi
- Başlık yeşil renkte ise tahmin doğru, kırmızı ise yanlıştır.

Bu görselleştirme, modelin hangi tür nesneleri kolay ayırt edebildiğini, hangilerinde ise zorlandığını gözlemlemeye olanak sağlar.

Görselleştirme kodundan bir parça:

```
plt.subplot(3, 5, i+1)
plt.imshow(last_15_images[i], cmap='gray')

pred_name = class_names[last_15_predictions[i]]
true_name = class_names[last_15_labels[i]]
```

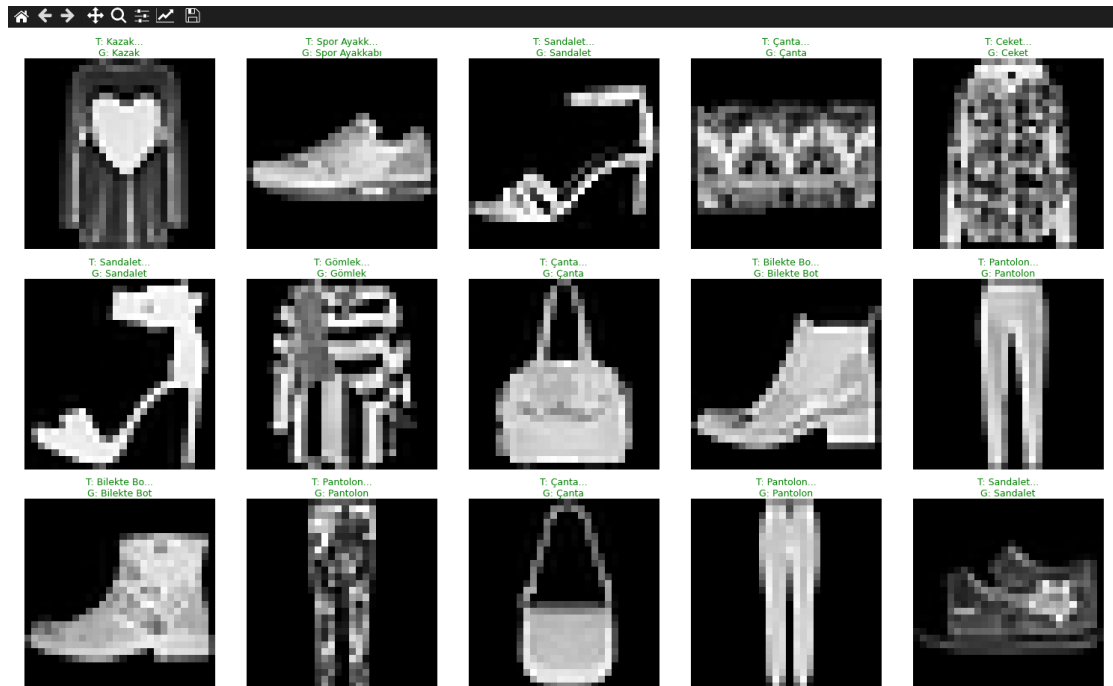
4.3. Metinsel Tahmin Tablosu

Görselleştirmeye ek olarak, son 15 tahmine ilişkin doğruluk durumları aşağıdaki gibi metin çıktısıyla da özetlenmiştir:

Son 15 Tahmin Sonucu:		
Resim 1:	✓	Kazak → Kazak
Resim 2:	✓	Spor Ayakkabı → Spor Ayakkabı
Resim 3:	✓	Sandalet → Sandalet
Resim 4:	✓	Çanta → Çanta
Resim 5:	✓	Ceket → Ceket
Resim 6:	✓	Sandalet → Sandalet
Resim 7:	✓	Gömlek → Gömlek
Resim 8:	✓	Çanta → Çanta
Resim 9:	✓	Bilekte Bot → Bilekte Bot
Resim 10:	✓	Pantolon → Pantolon
Resim 11:	✓	Bilekte Bot → Bilekte Bot
Resim 12:	✓	Pantolon → Pantolon
Resim 13:	✓	Çanta → Çanta
Resim 14:	✓	Pantolon → Pantolon
Resim 15:	✓	Sandalet → Sandalet

4.4. Başarı ve Limitasyonlar

Model basit bir MLP yapısına sahip olmasına rağmen; normalize edilmiş veriler, uygun ağırlık başlatma ve temel gradyan inişi ile %85 civarında doğruluk elde etmiştir. Ancak bazı sınıflar birbirine çok benzediğinden (örneğin, “Gömlek” ve “Ceket”), karışıklık yaşanmıştır. Bu gibi durumlar, daha gelişmiş yöntemlerle (örneğin CNN kullanımı) iyileştirilebilir.



5. Sonular, Tartışma ve Gelecek alıřmalar

5.1. Genel Deęerlendirme

Bu projede, Fashion MNIST veri seti kullanılarak temel bir yapay sinir aęı (MLP) ile grsel sınıflandırma problemi zlmřtr. Model yalnızca tek gizli katman ve sigmoid aktivasyon fonksiyonları ile oluřturulmuř; TensorFlow gibi geliřmiř derin ęrenme araları yerine NumPy tabanlı manuel bir yaklařım benimsenmiřtir. Bu sayede sinir aęlarının matematiksel temelleri, ęrenme sreleri ve eęitim adımları daha derinlemesine anlařılmıřtır.

Elde edilen doęruluk oranı %84 civarındadır. Bu oran, basit bir mimari ve sınırlı katman derinlięine raęmen tatmin edici dzeydedir.

5.2. Elde Edilen Bařarılar

- Veri n iřleme, one-hot encoding, aęırlık bařlatma, ileri ve geri yayılım sreleri bařarıyla uygulanmıřtır.
- Mini-batch yntemi ile modelin daha stabil ve hızlı ęrenmesi saęlanmıřtır.
- Grselleřtirme sayesinde modelin tahmin performansı sezgisel olarak da gzlemlenmiřtir.
- Manuel kodlama sayesinde sinir aęı eęitiminin tm matematiksel sreleri somut olarak kavranmıřtır.

5.3. Karřılařılan Zorluklar

- Sigmoid fonksiyonunun ıkıřlarda doęgunluęa girme (saturasyon) problemi, ęrenmeyi zaman zaman yavařlatmıřtır.
- ıkıř katmanında softmax yerine sigmoid kullanılması, ok sınıflı sınıflandırmalarda ideal olmayan sonulara yol aabilmıřtır.
- “Gmlek” ve “Ceket” gibi grsel olarak benzer sınıflar arasında modelin kararsız kaldıęı gzlemlenmiřtir.

Bu zorluklara raęmen genel bařarı oranı yksektir ve model ęrenmeyi bařarmıřtır.

5.4. Gelecekteki Geliřtirme Fikirleri

Projenin ilerleyen ařamalarında řu geliřtirmeler planlanabilir:

- **Softmax + Cross-Entropy** kombinasyonunun kullanılması, ok sınıflı problemler iin daha uygun olur.
- Gizli katman sayısı ve nron sayısı artırılarak modelin kapasitesi geniřletilebilir.
- Sigmoid yerine **ReLU** gibi daha gncel aktivasyon fonksiyonları kullanılabilir.
- Eęitim sreci hızlandırmak iin **momentum**, **Adam optimizer** gibi geliřmiř optimizasyon yntemleri entegre edilebilir.
- Grsel veri iřleme iin daha uygun olan **Convolutional Neural Network (CNN)** yapılarıyla doęruluk oranı ciddi řekilde artırılabilir.
- Eęitim ve test doęrulukları zamana gre izdirilerek overfitting gzlemlenebilir.

5.5. Sonu

Bu proje, grnt sınıflandırma alanında yapay sinir ağılarının temel yapıtaşlarının uygulamalı olarak ğrenilmesini saėlamıştır. Geliştirilen model, basit yapısına rağmen yksek doėruluk oranlarına ulaşmış ve başarılı tahminler gerekleştirmiştir. Elde edilen sonuçlar, yapay zekâ temelli modellemenin yalnızca gelişmiş ktphanelerle deėil, temel programlama teknikleriyle de etkili şekilde yapılabileceėini gstermektedir.

Bu alıřma, hem algoritma mantıėını ğrenmek isteyen ėrenciler hem de sinir aėına giriş yapmak isteyen kiřiler iin deėerli bir rnek niteliėindedir.

6. Kaynakça:

<https://arxiv.org/pdf/1708.07747>

https://cnedwards.com/files/Edwards_Yen_Final_Project.pdf

https://www.cs.toronto.edu/~rgrosse/courses/csc321_2017/slides/lec5.pdf

<https://www.cs.upc.edu/~mmartin/ml-mds/ml-06.pdf>

https://hpi.de/fileadmin/user_upload/fachgebiete/meinel/team/Haojin/competitive_problem_solving_with_deep_learning/Implementing_a_Neural_Network_from_Scratch.pdf

https://sebastianraschka.com/pdf/books/dlb/appendix_f_numpy-intro.pdf

<https://statisticalhorizons.com/wp-content/uploads/2024/02/DLP-Sample-Materials-1.pdf>

<https://www.ijeast.com/papers/310-316%2CTesma412%2CIJEAST.pdf>

<https://proceedings.mlr.press/v9/glorot10a/glorot10a.pdf>

https://cs230.stanford.edu/fall2018/section_files/section5_handout.pdf

<https://proceedings.mlr.press/v202/mao23b/mao23b.pdf>

<https://www.niser.ac.in/~smishra/teach/cs460/23cs460/lectures/lec23.pdf>

<https://cklixx.people.wm.edu/teaching/math400/Annette-paper.pdf>

https://www.cs.toronto.edu/~rgrosse/courses/csc321_2018/readings/L06%20Backpropagation.pdf

<https://cs230.stanford.edu/files/C2M2.pdf>

<https://www.cs.cornell.edu/courses/cs6787/2019fa/lectures/Lecture3.pdf>

https://www.tutorialspoint.com/matplotlib/matplotlib_tutorial.pdf

https://www.mat.uniroma2.it/~speleers/teaching/labcalc/slides_python_3.pdf

https://jingyuexing.github.io/Ebook/Machine_Learning/Neural%20Networks%20and%20Deep%20Learning-eng.pdf

<https://d2l.ai/d2l-en.pdf>